

**Министерство науки и высшего образования Российской Федерации**  
Федеральное государственное автономное образовательное учреждение  
высшего образования

**«Пермский национальный исследовательский политехнический  
университет»**

Электротехнический факультет

Выпускающая кафедра: Информационные технологии и  
автоматизированные системы (ИТАС)

Направление подготовки: 09.03.04 Программная инженерия

## **ОТЧЕТ**

Лабораторная работа №13

«Стандартные обобщенные алгоритмы библиотеки STL.»

**По дисциплине «Основы алгоритмизации и программирования»**

Вариант 15

Выполнил: студент группы РИС-25-2Б

Ильинов Фёдор Михайлович

Принял: Доц. Полякова О. А.

Пермь 2026

## ПОСТАНОВКА ЗАДАЧИ

### Задача 1.

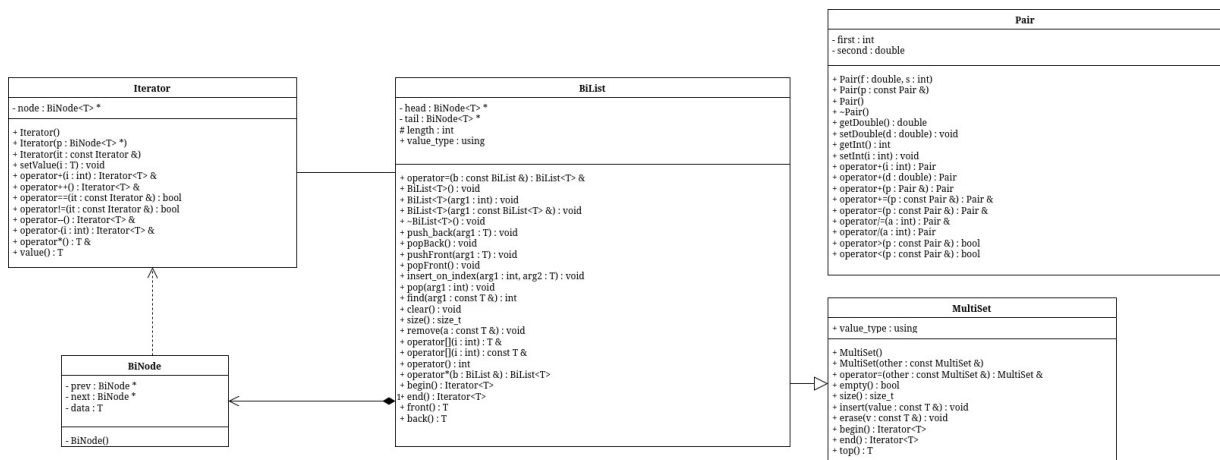
1. Создать последовательный контейнер.
2. Заполнить его элементами пользовательского типа (тип указан в варианте). Для пользовательского типа перегрузить необходимые операции.
3. Заменить элементы в соответствии с заданием (использовать алгоритмы `replace_if()`, `replace_copy()`, `replace_copy_if()`, `fill()`).
4. Удалить элементы в соответствии с заданием (использовать алгоритмы `remove()`, `remove_if()`, `remove_copy_if()`, `remove_copy()`)
5. Отсортировать контейнер по убыванию и по возрастанию ключевого поля (использовать алгоритм `sort()`).
6. Найти в контейнере заданный элемент (использовать алгоритмы `find()`, `find_if()`, `count()`, `count_if()`).
7. Выполнить задание варианта для полученного контейнера (использовать алгоритм `for_each()`).
8. Для выполнения всех заданий использовать стандартные алгоритмы библиотеки STL. Задача 2.
9. Создать адаптер контейнера.
10. Заполнить его элементами пользовательского типа (тип указан в варианте). Для пользовательского типа перегрузить необходимые операции.
11. Заменить элементы в соответствии с заданием (использовать алгоритмы `replace_if()`, `replace_copy()`, `replace_copy_if()`, `fill()`).
12. Удалить элементы в соответствии с заданием (использовать алгоритмы `remove()`, `remove_if()`, `remove_copy_if()`, `remove_copy()`)

13. Отсортировать контейнер по убыванию и по возрастанию ключевого поля (использовать алгоритм `sort()`).
14. Найти в контейнере элемент с заданным ключевым полем (использовать алгоритмы `find()`, `find_if()`, `count()`, `count_if()`).
15. Выполнить задание варианта для полученного контейнера (использовать алгоритм `for_each()`).
16. Для выполнения всех заданий использовать стандартные алгоритмы библиотеки STL. Задача 3
17. Создать ассоциативный контейнер.
18. Заполнить его элементами пользовательского типа (тип указан в варианте). Для пользовательского типа перегрузить необходимые операции.
19. Заменить элементы в соответствии с заданием (использовать алгоритмы `replace_if()`, `replace_copy()`, `replace_copy_if()`, `fill()`).
20. Удалить элементы в соответствии с заданием (использовать алгоритмы `remove()`, `remove_if()`, `remove_copy_if()`, `remove_copy()`).
21. Отсортировать контейнер по убыванию и по возрастанию ключевого поля (использовать алгоритм `sort()`).
22. Найти в контейнере элемент с заданным ключевым полем (использовать алгоритмы `find()`, `find_if()`, `count()`, `count_if()`).
23. Выполнить задание варианта для полученного контейнера (использовать алгоритм `for_each()`).
24. Для выполнения всех заданий использовать стандартные алгоритмы библиотеки STL.

## АНАЛИЗ.

1. Воспользуемся функциями из работы №11, изменяя логику там, где это возможно, на алгоритмы STL.
2. Добавим вывод отсортированного контейнера по возрастанию (asc) и по убыванию (desc) для демонстрации работы функции sort().
3. Для работы с for\_each воспользуемся лямбда-функциями, используя захват по ссылке для изменения необходимых параметров.

## UML-ДИАГРАММА



## КОД Pair.h

```
#ifndef ILINOV_FEDOR_RIS_25_2B_LABS_PSTU_CS_FRACTION_H
#define ILINOV_FEDOR_RIS_25_2B_LABS_PSTU_CS_FRACTION_H
#include <ostream>

class Pair {
    int first;
    double second;

public:
    Pair(double, int);
    Pair(const Pair &);
    Pair();
    ~Pair();
    double getDouble() const;
    void setDouble(double);
    int getInt() const;
    void setInt(int);
    Pair &operator=(Pair const &p);
    Pair operator+(Pair &p);
    Pair operator+(int);
    Pair operator+(double);
    Pair& operator+=(Pair const &p);
    Pair& operator/=(int a);
    Pair operator/(int a) const;
    bool operator>(Pair const &p) const;
    bool operator<(Pair const &p) const;
    bool operator==(const Pair &p) const;
    friend std::ostream &operator<<(std::ostream &, const Pair &);
    friend std::istream &operator>>(std::istream &, Pair &);
};

#endif //ILINOV_FEDOR_RIS_25_2B_LABS_PSTU_CS_FRACTION_H
```

## КОД Pair.cpp

```
#include "Pair.h"
#include <iostream>

Pair::Pair(double a, int b) {
    first = b;
    second = a;
}

Pair::Pair(const Pair &f) {
    second = f.second;
    first = f.first;
}

Pair::Pair() {
    second = 0;
    first = 0;
}

Pair::~Pair() {
    second = 0;
    first = 0;
}

double Pair::getDouble() const {
    return second;
}

void Pair::setDouble(double a) {
    second = a;
}

int Pair::getInt() const {
    return first;
}

void Pair::setInt(int a) {
    first = a;
}

Pair &Pair::operator=(Pair const &p) {
    second = p.second;
    first = p.first;
    return *this;
}
```

```
}
```

```
bool Pair::operator<(Pair const &p) const {  
    if (this->getInt() < p.getInt())  
        return true;  
    if (this->getInt() == p.getInt())  
        return this->getDouble() < p.getDouble();  
    return false;  
}
```

```
bool Pair::operator>(Pair const &p) const {  
    return p < *this;  
}
```

```
Pair & Pair::operator+=(Pair const &p) {  
    this->first += p.first;  
    this->second += p.second;  
    return *this;  
}
```

```
Pair & Pair::operator/=(int a) {  
    this->first /= a;  
    this->second /= a;  
    return *this;  
}
```

```
Pair Pair::operator/(int a) const {  
    return Pair(  
        this->second / a,  
        this->first / a  
    );  
}
```

```
bool Pair::operator==(const Pair &p) const {  
    return (this->first == p.first && this->second == p.second);  
}
```

```
std::ostream &operator<<(std::ostream &stream, const Pair &p) {  
    std::cout << p.first << ":" << p.second;  
    return stream;  
}
```

```
std::istream &operator>>(std::istream &stream, Pair &p) {  
    std::cout << "Double? ";  
    stream >> p.second;  
}
```

```
std::cout << "Int? ";  
stream >> p.first;  
return stream;  
}
```

```
Pair Pair::operator+(Pair &p) {  
    Pair second(*this);  
    second.second += p.second;  
    second.first += p.first;  
    return second;  
}
```

```
Pair Pair::operator+(int a) {  
    Pair tmp(*this);  
    tmp.first += a;  
    return tmp;  
}
```

```
Pair Pair::operator+(double a) {  
    Pair tmp(*this);  
    tmp.second += a;  
    return tmp;  
}
```



## КОД main.cpp

```
#include <iostream>
#include <list>
#include <map>
#include <queue>
#include <random>
#include <algorithm>
#include "Pair.h"
#include "PriorityQueue.h"

std::random_device rd;
std::mt19937 gen(rd());

template<class T>
void printItem(const T& i) { std::cout << i << ' '; }

void printPair(const std::pair<const Pair, int>& pair) { std::cout << "[" << pair.first << "]" = " <<
pair.second << " "; }

Pair extractKey(const std::pair<const Pair, int>& pair) { return pair.first; }

template<class T>
void printContainerAfterSubtask(const T& l) {
    std::cout << "\t\t";
    std::for_each(l.begin(), l.end(), printItem<typename T::value_type>);
    std::cout << '\n';
}

template<class T>
T subtask1(const T& t) {
    T l = t;
    std::cout << "\tsubtask 1\n";
    using Item = typename T::value_type;

    Item avg = Item();
    std::for_each(l.begin(), l.end(), [&avg](const Item& i){ avg += i; });
    avg /= (int)l.size();
    l.push_back(avg);

    std::cout << "\t\t" << "avg: " << avg << '\n';

    return l;
}
```

```

template<class T>
T subtask2(const T& t) {
    T l = t;

    std::cout << "\tsubtask 2\n";
    using Item = typename T::value_type;

    std::uniform_int_distribution<> dist(0, 2);

    std::vector<Item> forDelete;
    std::remove_copy_if(l.begin(), l.end(), std::back_inserter(forDelete),
        [&](const Item&){ return dist(gen) != 0; });

    std::cout << "\t\tFor delete:\n\t\t";

    std::for_each(forDelete.begin(), forDelete.end(), printItem<Item>);

    if constexpr (std::is_same_v<T, std::list<Item>>) {
        std::for_each(forDelete.begin(), forDelete.end(), [&](const Item& i){ l.remove(i); });
    } else {
        std::for_each(forDelete.begin(), forDelete.end(), [&](const Item& i){
            l.erase(std::remove(l.begin(), l.end(), i), l.end());
        });
    }
    std::cout << '\n';

    return l;
}

```

```

template<class T>
T subtask3(const T& t) {
    T l = t;

    std::cout << "\tsubtask 3\n";
    using Item = typename T::value_type;

    auto mn = *std::min_element(l.begin(), l.end());
    auto mx = *std::max_element(l.begin(), l.end());
    Item avg = (mn + mx) / 2;

    std::for_each(l.begin(), l.end(), [&avg](Item& i){ i += avg; });

    std::cout << "\t\t(min + max) / 2 = " << avg << '\n';

    return l;
}

```

```

}

void task_1() {
    std::cout << "task 1\n";

    std::list<double> l;
    std::uniform_int_distribution<> dist(-99, 99);
    for (int i = 0; i < 10; i++)
        l.push_back((double) dist(gen) / 10);

    std::cout << '\t';
    std::for_each(l.begin(), l.end(), printItem<double>);
    std::cout << '\n';

    l.sort();
    std::cout << "\t[asc]: "; std::for_each(l.begin(), l.end(), printItem<double>); std::cout << '\n';
    l.sort(std::greater<double>());
    std::cout << "\t[desc]: "; std::for_each(l.begin(), l.end(), printItem<double>); std::cout << '\n';

    l = subtask1(l);
    printContainerAfterSubtask(l);

    l = subtask2(l);
    printContainerAfterSubtask(l);

    l = subtask3(l);
    printContainerAfterSubtask(l);
}

template<class T>
PriorityQueue<T> copyBiListToSTLPriorityQueue(const std::vector<T>& bl) {
    PriorityQueue<T> q;
    std::for_each(bl.begin(), bl.end(), [&q](const T& i){ q.push(i); });
    return q;
}

template<class T>
std::vector<T> copySTLPriorityQueueToBiList(PriorityQueue<T>& q) {
    std::vector<T> bl;
    while (!q.empty()) {
        bl.push_back(q.top());
        q.pop();
    }
    q = copyBiListToSTLPriorityQueue(bl);
    return bl;
}

```

```
}
```

```
template<class T>
void printSTLPriorityQueue(PriorityQueue<T>& q) {
    auto v = copySTLPriorityQueueToBiList(q);
    std::for_each(v.begin(), v.end(), printItem<T>);
    std::cout << '\n';
}
```

```
void task_2() {
    std::cout << "task 2\n";
    PriorityQueue<Pair> q;

    std::uniform_int_distribution<> dist(0, 9);
    for (int i = 0; i < 10; i++) {
        Pair p((double)dist(gen) / 10, dist(gen));
        q.push(p);
    }

    std::cout << '\t';
    printSTLPriorityQueue(q);

    {
        auto sv = copySTLPriorityQueueToBiList(q);
        std::sort(sv.begin(), sv.end());
        std::cout << "\t[asc]: "; std::for_each(sv.begin(), sv.end(), printItem<Pair>); std::cout << '\n';
        std::sort(sv.begin(), sv.end(), std::greater<Pair>());
        std::cout << "\t[desc]: "; std::for_each(sv.begin(), sv.end(), printItem<Pair>); std::cout << '\n';
    }

    auto v = copySTLPriorityQueueToBiList(q);
    v = subtask1(v);
    q = copyBiListToSTLPriorityQueue(v);
    std::cout << "\t\t";
    printSTLPriorityQueue(q);

    v = copySTLPriorityQueueToBiList(q);
    v = subtask2(v);
    q = copyBiListToSTLPriorityQueue(v);
    std::cout << "\t\t";
    printSTLPriorityQueue(q);

    v = copySTLPriorityQueueToBiList(q);
    v = subtask3(v);
    q = copyBiListToSTLPriorityQueue(v);
```

```

std::cout << "\t\t";
printSTLPriorityQueue(q);

std::cout << '\n';
}

void task_3() {
    std::cout << "task 3\n";
    std::map<Pair, int> mp;

    std::uniform_int_distribution<> dist(0, 9);
    for (int i = 0; i < 10; i++) {
        Pair p((double)dist(gen) / 10, dist(gen));
        mp[p] = dist(gen);
    }

    std::cout << '\t';
    std::for_each(mp.begin(), mp.end(), printPair);
    std::cout << '\n';

    std::vector<Pair> v;
    std::transform(mp.begin(), mp.end(), std::back_inserter(v), extractKey);

    std::sort(v.begin(), v.end());
    std::cout << "\t[asc]: "; std::for_each(v.begin(), v.end(), printItem<Pair>); std::cout << '\n';
    std::sort(v.begin(), v.end(), std::greater<Pair>());
    std::cout << "\t[desc]: "; std::for_each(v.begin(), v.end(), printItem<Pair>); std::cout << '\n';

    v = subtask1(v);
    std::cout << "\t\t";
    std::for_each(v.begin(), v.end(), printItem<Pair>);
    std::cout << '\n';

    v = subtask2(v);
    std::cout << "\t\t";
    std::for_each(v.begin(), v.end(), printItem<Pair>);
    std::cout << '\n';

    v = subtask3(v);
    std::cout << "\t\t";
    std::for_each(v.begin(), v.end(), printItem<Pair>);
    std::cout << '\n';

    std::cout << '\n';
}

```

```
int main() {
    task_10;
    task_20;
    task_30;
}
```

КОД bidirectional.hpp

```
#ifndef ILINOV_FEDOR_RIS_25_2B_LABS_PSTU_CS_BI_H
#define ILINOV_FEDOR_RIS_25_2B_LABS_PSTU_CS_BI_H
#include <iosfwd>
```

```
template<class T>
class BiList;
```

```
template<class T>
std::ostream &operator<<(std::ostream &, BiList<T>);
```

```
template<class T>
std::istream &operator>>(std::istream &, BiList<T> &);
```

```
template<class T>
class Iterator;
```

```
template<class T>
class BiNode {
    friend BiList<T>;
    friend Iterator<T>;
    BiNode *prev;
    BiNode *next;
    T data;
```

```
    BiNode() { prev = nullptr, next = nullptr, data = T(); }
};
```

```
template<class T>
class Iterator {
    BiNode<T> *node;
```

```
public:
    Iterator() { node = nullptr; }
    Iterator(BiNode<T> *p) { node = p; }
    Iterator(const Iterator &it) { node = it.node; }
    bool operator==(const Iterator &it) { return node == it.node; }
    bool operator!=(const Iterator &it) { return node != it.node; };
```

```

Iterator<T> &operator++();

Iterator<T> &operator--();

Iterator<T> &operator+(int);

Iterator<T> &operator-(int);

T &operator*() { return node->data; }
T value() { return node->data; }
void setValue(T i) { node->data = i; }
};

template<class T>
class BiList {
    BiNode<T> *head;
    BiNode<T> *tail;

protected:
    int length = 0;
public:
    using value_type = T;

    BiList<T>();

    BiList<T>(int);

    BiList<T>(const BiList<T> &);

    ~BiList<T>() { clear(); }

    void push_back(T);

    void popBack();

    void pushFront(T);

    void popFront();

    void insert_on_index(int, T);

    void pop(int);

    int find(const T &);

```

```

void clear();

size_t size() const;

void remove(const T &a);

T &operator[](int);

const T &operator[](int) const;

BiList<T> &operator=(const BiList &);

int operator()() const;

BiList<T> operator*(BiList &);

Iterator<T> begin() const { return Iterator(head); }
Iterator<T> end() const { return Iterator(tail ? tail->next : nullptr); }

T front();

T back();
};

#include "bidirectional.hpp"
#include <iostream>
#include <fstream>

template<class T>
void BiList<T>::push_back(T data) {
    auto node = new BiNode<T>;
    node->data = data;
    if (length == 0) {
        head = node;
        tail = node;
        length++;
        return;
    }

    node->prev = tail;
    tail->next = node;
    tail = node;
    length++;
}

```



```
template<class T>
void BiList<T>::popBack() {
    if (length == 0)
        return;

    auto del = tail;
    tail = tail->prev;

    if (tail != nullptr)
        tail->next = nullptr;
    else
        head = nullptr;

    delete del;
    length--;
}
```

```
template<class T>
void BiList<T>::pushFront(T data) {
    auto node = new BiNode<T>;
    node->data = data;
    if (length == 0) {
        head = node;
        tail = node;
        length++;
        return;
    }

    node->next = head;
    head->prev = node;
    head = node;
    length++;
}
```

```
template<class T>
void BiList<T>::popFront() {
    if (length == 0)
        return;

    auto del = head;
    head = head->next;
```

```

if (head != nullptr)
    head->prev = nullptr;
else
    tail = nullptr;

delete del;
length--;
}

```

```

template<class T>
void BiList<T>::insert_on_index(int idx, T data) {
    if (idx == length) {
        push_back(data);
        return;
    }
    if (idx == 0) {
        pushFront(data);
        return;;
    }

    if (!(0 < idx && idx < length)) {
        return;
    }

    auto node = head;
    for (int i = 0; i < idx; i++) {
        node = node->next;
    }

    auto newNode = new BiNode<T>;
    newNode->data = data;

    node->prev->next = newNode;
    newNode->prev = node->prev;

    node->prev = newNode;
    newNode->next = node;

    length++;
}

```

```

template<class T>
void BiList<T>::pop(int idx) {

```

```

    if (!(0 <= idx && idx < length)) {
        return;
    }
    if (idx == 0) {
        popFront();
        return;
    }
    if (idx == length - 1) {
        popBack();
        return;
    }

    auto node = head;
    for (int i = 0; i < idx; i++) {
        node = node->next;
    }

    node->next->prev = node->prev;
    node->prev->next = node->next;

    delete node;
    length--;
}

```

```

template<class T>
int BiList<T>::find(const T &data) {
    auto node = head;
    int i = 0;
    while (node != nullptr) {
        if (node->data == data) {
            return i;
        }
        i++;
        node = node->next;
    }

    return -1;
}

```

```

template<class T>
void BiList<T>::clear() {
    auto node = head;
    while (node != nullptr) {
        auto tmp = node->next;

```

```

        delete node;
        node = tmp;
    }
    head = nullptr;
    tail = nullptr;
    length = 0;
}

```

```

template<class T>
size_t BiList<T>::size() const {
    return length;
}

```

```

template<class T>
void BiList<T>::remove(const T &a) {
    while (find(a) != -1)
        pop(find(a));
}

```

```

template<class T>
T &BiList<T>::operator[](int idx) {
    if (!(0 <= idx && idx < length)) {
        exit(1);
    }
}

```

```

    auto node = head;
    for (int i = 0; i < idx; i++) {
        node = node->next;
    }

    return node->data;
}

```

```

template<class T>
const T &BiList<T>::operator[](int idx) const {
    if (!(0 <= idx && idx < length)) {
        exit(1);
    }

    auto node = head;
    for (int i = 0; i < idx; i++) {
        node = node->next;
    }
}

```

```

    return node->data;
}

template<class T>
BiList<T> &BiList<T>::operator=(const BiList<T> &l) {
    if (this == &l)
        return *this;

    this->clear();
    for (int i = 0; i < l.size(); i++) {
        this->push_back(l[i]);
    }

    return *this;
}

template<class T>
BiList<T> BiList<T>::operator*(BiList &l) {
    BiList res;
    int range = std::min(l.size(), this->size());
    for (int i = 0; i < range; i++) {
        res.push_back((*this)[i] * l[i]);
    }
    return res;
}

template<class T>
T BiList<T>::front() {
    return head->data;
}

template<class T>
T BiList<T>::back() {
    return tail->data;
}

template<class T>
BiList<T>::BiList() {
    head = nullptr;
    tail = nullptr;
    length = 0;
}

template<class T>

```

```

BiList<T>::BiList(int s) {
    head = nullptr;
    tail = nullptr;
    length = 0;
    for (int i = 0; i < s; i++)
        push_back(T());
}

```

```

template<class T>
BiList<T>::BiList(const BiList<T> &l) {
    head = nullptr;
    tail = nullptr;
    length = 0;
    for (int i = 0; i < l.size(); i++) {
        push_back(l[i]);
    }
}

```

```

template<class T>
std::ostream &operator<<(std::ostream &stream, BiList<T> l) {
    for (int i = 0; i < l(); i++)
        stream << l[i] << '\t';
    return stream;
}

```

```

template<class T>
std::istream &operator>>(std::istream &stream, BiList<T> &l) {
    l.clear();
    std::cout << "Size? ";
    int s;
    stream >> s;
    std::cout << "Elements? ";
    for (int i = 0; i < s; i++) {
        T a;
        stream >> a;
        l.push_back(a);
    }
    return stream;
}

```

```

template<class T>
Iterator<T> &Iterator<T>::operator+(int a) {
    for (int i = 0; i < a; i++)
        node = node->next;
    return *this;
}

```

```
}
```

```
template<class T>
Iterator<T> &Iterator<T>::operator-(int a) {
    for (int i = 0; i < a; i++)
        node = node->prev;
    return *this;
}
```

```
template<class T>
Iterator<T> &Iterator<T>::operator--() {
    node = node->prev;
    return *this;
}
```

```
template<class T>
Iterator<T> &Iterator<T>::operator++() {
    node = node->next;
    return *this;
}
```

```
#endif //ILINOV_FEDOR_RIS_25_2B_LABS_PSTU_CS_BI_H
```

#### КОД PriorityQueue.h

```
#ifndef ILINOV_FEDOR_RIS_25_2B_LABS_PSTU_CS_PRIORITYQUEUE_H
#define ILINOV_FEDOR_RIS_25_2B_LABS_PSTU_CS_PRIORITYQUEUE_H
#include "bidirectional.hpp"
```

```
template<class T>
class PriorityQueue : private BiList<T>{
public:
    PriorityQueue() : BiList<T>({});
    PriorityQueue(const PriorityQueue& other) : BiList<T>(other){};

    PriorityQueue& operator=(const PriorityQueue& other) {
        BiList<T>::operator=(other);
        return *this;
    };

    bool empty() const {
        return !BiList<T>::size();
    };

    size_t size() const {
```

```

    return BiList<T>::size();
};

T top() {
    T mx = BiList<T>::front();
    for (auto& i : *this)
        if (i > mx) mx = i;
    return mx;
};

void push(const T& value) {
    BiList<T>::push_back(value);
};

void pop() {
    BiList<T>::remove(top());
};
};

```

*#endif //ILINOV\_FEDOR\_RIS\_25\_2B\_LABS\_PSTU\_CS\_PRIORITYQUEUE\_H*

## РЕЗУЛЬТАТ ВЫПОЛНЕНИЯ

/home/localuser/CLionProjects/Ilinov\_Fedor\_RIS\_25\_2B\_Labs\_PSTU\_CS/sem2/manual\_2\_oop/  
lab13/main

task 1

2.4 8.9 -1.4 -2 -1.1 -9.3 3.4 -3.6 -9.4 7.5

[asc]: -9.4 -9.3 -3.6 -2 -1.4 -1.1 2.4 3.4 7.5 8.9

[desc]: 8.9 7.5 3.4 2.4 -1.1 -1.4 -2 -3.6 -9.3 -9.4

subtask 1

avg: -0.46

8.9 7.5 3.4 2.4 -1.1 -1.4 -2 -3.6 -9.3 -9.4 -0.46

subtask 2

For delete:

8.9 2.4 -1.4

7.5 3.4 -1.1 -2 -3.6 -9.3 -9.4 -0.46

subtask 3



$$(\min + \max) / 2 = -0.95$$

6.55 2.45 -2.05 -2.95 -4.55 -10.25 -10.35 -1.41

task 2

9:0.7 8:0.1 5:0.2 4:0.4 4:0 3:0.8 3:0.6 2:0.9 2:0.6 1:0.9

[asc]: 1:0.9 2:0.6 2:0.9 3:0.6 3:0.8 4:0 4:0.4 5:0.2 8:0.1 9:0.7

[desc]: 9:0.7 8:0.1 5:0.2 4:0.4 4:0 3:0.8 3:0.6 2:0.9 2:0.6 1:0.9

subtask 1

avg: 4:0.52

9:0.7 8:0.1 5:0.2 4:0.52 4:0.4 4:0 3:0.8 3:0.6 2:0.9 2:0.6 1:0.9

subtask 2

For delete:

8:0.1 5:0.2

9:0.7 4:0.52 4:0.4 4:0 3:0.8 3:0.6 2:0.9 2:0.6 1:0.9

subtask 3

$$(\min + \max) / 2 = 5:0.8$$

14:1.5 9:1.32 9:1.2 9:0.8 8:1.6 8:1.4 7:1.7 7:1.4 6:1.7

task 3

(([0:0.6] = 6) ([1:0.6] = 1) ([2:0.8] = 1) ([3:0.7] = 4) ([3:0.9] = 9) ([6:0.1] = 4) ([7:0.4] = 7)  
([8:0.8] = 4) ([8:0.9] = 9) ([9:0.1] = 0)

[asc]: 0:0.6 1:0.6 2:0.8 3:0.7 3:0.9 6:0.1 7:0.4 8:0.8 8:0.9 9:0.1

[desc]: 9:0.1 8:0.9 8:0.8 7:0.4 6:0.1 3:0.9 3:0.7 2:0.8 1:0.6 0:0.6

subtask 1

avg: 4:0.59

9:0.1 8:0.9 8:0.8 7:0.4 6:0.1 3:0.9 3:0.7 2:0.8 1:0.6 0:0.6 4:0.59

subtask 2

For delete:

3:0.7 0:0.6 4:0.59

9:0.1 8:0.9 8:0.8 7:0.4 6:0.1 3:0.9 2:0.8 1:0.6

subtask 3

$$(\min + \max) / 2 = 5:0.35$$

14:0.45 13:1.25 13:1.15 12:0.75 11:0.45 8:1.25 7:1.15 6:0.95